

WEEK 6

Parallel Programming

HPC & Cluster Computing

Course: Operating Systems 4 (HPC)
Module: EIOSY4A
Topic: Parallel Programming (OpenMP and MPI)
Reading Time: ~20 Minutes

Learning Outcomes

By the end of this week, you should be able to:

1. **Evaluate** the architectural trade-offs between Shared Memory (OpenMP) and Distributed Memory (MPI) paradigms.
2. **Predict and analyze** the upper theoretical limits of parallel scaling using Amdahl's Law.
3. **Implement** OpenMP directives (parallel for, reduction, critical) to parallelize serial code blocks.
4. **Design** distributed memory applications using the MPI SPMD model, communicators, and point-to-point operations.
5. **Diagnose and resolve** complex concurrent programming flaws, including race conditions and deadlocks.

Week Goal

Welcome to Week 6! We now switch roles from System Administrator to HPC Software Developer. This week, you learn why a cluster is useless if software runs serially, and how to unlock its power through parallel programming the foundational skill of HPC.

WEEK 5 WEEK 7

Assessment Alignment: The weekly quiz and practical assignments for this module specifically evaluate the Learning Outcomes listed below, verifying your ability to operate at an NQF Level 7 standard of analysis and synthesis.

Overview

In the first four weeks, we built the physical and logical infrastructure of a cluster (nodes, network, scheduler). Now, we switch roles from **Systems Administrator** to **HPC Software Developer**.

A cluster is useless if software runs serially on a single core. To unlock the power of hundreds of cores, we must rewrite applications to execute in parallel. This week introduces the theoretical foundations of parallelism and the two fundamental paradigms: **Shared Memory** (using OpenMP) and **Distributed Memory** (using MPI).

This content covers **Sterling Chapters 7 (Concepts), 8 (Shared Memory), and 9 (Distributed Memory)**.

---

1. Foundations of Parallelism (Sterling Chapter 7)

Before writing code, we must understand the *physics* of performance.

1.1 Speedup and Efficiency

The primary goal of parallel computing is **Speedup**.

- **Speedup (S)**: The ratio of the time taken to solve a problem on a single processor (T_1) to the time taken on p processors.

- If 10% of your code is serial ($f = 0.1$), the maximum theoretical speedup is 10x, even with infinite processors.
- **Implication**: You must minimize the serial bottleneck (I/O, initialization, single-threaded reductions).

1.3 Gustafson's Law (Weak Scaling)

Gustafson's Law is the optimistic view. It assumes that as you get more processors, you will **increase the problem size** rather than just trying to solve the same problem faster.

- If we have 1000x more CPUs, we run a simulation with 1000x more particles.
- The parallel part grows, diluting the serial part.

---

2. Shared Memory Paradigm: OpenMP (Sterling Chapter 8)

In a **Shared Memory** system (like a modern 64-core AMD EPYC server), all processors can read and write to the same RAM. This is the "easiest" way to parallelize because you don't need to move data.

OpenMP (Open Multi-Processing) is the industry standard for shared-memory programming. It uses **Compiler Directives** (`#pragma`) to instruct the compiler to spawn threads.

2.1 The Fork-Join Model

OpenMP programs follow a

1. **Master**: Program starts as a single thread (master).

2. **Fork**: Master thread creates a parallel region (`#pragma omp parallel`) and splits into a Team of threads.
3. **Execute**: Threads execute the block simultaneously.
4. **Join**: Threads finish and merge back into the Master thread (Implicit Barrier).

2.2 Core Directives and Clauses

You control OpenMP behavior using clauses appended to the directive.

1. Parallelizing Loops (`#pragma omp parallel for`) The compiler splits the loop iterations among threads.

```
#pragma omp parallel for schedule(static)
for(int i=0; i<1000000; i++) {
    A[i] = B[i] + C[i];
}
#pragma omp parallel for schedule(static)
for(int i=0; i<1000000; i++) {
    A[i] = B[i] + C[i];
}
```

Explanation: The `#pragma` tells the compiler to divide the 1,000,000 iterations of the loop among the available threads. `schedule(static)` ensures that if there are 4 threads, each thread gets exactly 250,000 iterations, minimizing overhead.

2. Data Scoping (`shared` vs `private`)

- **Shared**: All threads access the exact same memory address. (Default for variables outside the parallel block).
- **Private**: Each thread gets its own uninitialized copy of the variable.
- **Firstprivate**: Each thread gets its own copy, initialized with the master's value.

3. Reductions (`reduction(op:list)`) How do you sum an array in parallel without a race condition? Each thread creates its own local private accumulator, and OpenMP

```
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)
for(int i=0; i<N; i++) {
    sum += A[i];
}
double sum = 0.0;
#pragma omp parallel for reduction(+:sum)
for(int i=0; i<N; i++) {
    sum += A[i];
}
```

Explanation: Without `reduction`, multiple threads would try to update `sum` simultaneously, causing a race condition. The `reduction(+:sum)` clause gives each thread a private copy of `sum` to accumulate its local total. When all threads finish, OpenMP correctly adds these private totals to the global `sum` variable.

2.3 Race Conditions and Synchronization

A **Race Condition** occurs when two threads try to write to the same shared variable simultaneously.

- **Example**: `count++` (Read, Modify, Write). If two threads read "5" at the same time, both write "6", losing one increment.

Fixes:

- `#pragma omp atomic`: Hardware-level protection for simple updates (`x += 1`). Fast.
- `#pragma omp critical`: Creates a "Critical Section" where only one thread can exist at a time. Slow (serializes code).
- `#pragma omp barrier`: All threads wait here until the last one arrives.

3. Distributed Memory Paradigm: MPI (Sterling Chapter 3)

OpenMP is limited to a single box (Scale-Up). To use a cluster (Scale-Out), we need **Distributed Memory**. In a cluster, Node A cannot access Node B's RAM. They share nothing. To exchange data, they must send **Messages** over the Network.

MPI (Message Passing Interface) is the standard specification for this. It is a library (not part of the compiler).

3.1 The SPMD Model

MPI programs are built on the **Single Program, Multiple Data (SPMD)** model. In this paradigm, you write and compile a single executable program (e.g., `main.c`). When you run this program on the cluster, the MPI runtime launches N identical copies of it across the available nodes. Each copy is referred to as a **Process**

Although every process runs the exact same code, they do not necessarily do the same thing at the same time. Their behavior diverges based on their unique identity, allowing one process to act as a manager while others act as workers, or for each process to operate on a different slice of a large dataset. This structure simplifies development because you only maintain one codebase, yet it allows for highly complex, uncoupled parallel behavior.

3.1.1 SPMD Code Example

Here is a canonical "Hello World" example demonstrating the SPMD model, as typically presented in *Introduction to High Performance Computing for Scientists and Engineers* (Hager & Wellein).

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
```

WEEK 5 WEEK 7

```
    MPI_Init(&argc, &argv);

    // 2. Determine the rank of the calling process in th
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // 3. Determine the size of the group associated with
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // 4. Each process executes this code, but behavior c
    if (rank == 0) {
        printf("Hello from the Master process (Rank %d of
    } else {
        printf("Hello from Worker process %d of %d\n", ra
    }

    // 5. Terminate the MPI execution environment
    MPI_Finalize();
    return 0;
}
```

Explanation: This example illustrates the fundamental MPI boilerplate. We declare `rank` (my ID) and `size` (total processes). `MPI_Init` starts the parallel environment. `MPI_Comm_rank` and `MPI_Comm_size` fill our variables. The `if (rank == 0)` block allows the Master process to separate its logic from the Workers, even though they all run the same compiled executable.

3.2 Ranks and Communicators

To enable collaboration, every process needs to know "Who am I?" and "Who can I talk to?". This is handled by **Ranks** and **Communicators**

A **Communicator** is a defined group of processes that participate in a common parallel task. The default communicator `MPI_COMM_WORLD` includes every active process in the job. Within a communicator, each process is assigned a unique integer identifier called a **Rank**. Ranks act like addresses for mailing messages. They start at 0 and go up to

while Ranks 1 through $N - 1$ serve as compute workers.

3.3 The Message Envelope

In a shared memory system, you share data by simply passing a pointer. In distributed memory, this is impossible because Node A cannot see Node B's RAM. Instead, data must be explicitly packed into a message and transmitted over the network. Every MPI message must be wrapped in a "Message Envelope" containing specific metadata so the receiver knows how to handle it.

The envelope consists of four critical pieces of information for routing: the **Source** (who sent it?), the **Destination** (who is it for?), the **Tag**, and the **Communicator**. The **Tag** is an arbitrary integer label used to distinguish between different types of messages; for example, you might use Tag `0` for sending raw data and Tag `1` for sending control signals like "terminate."

Inside the envelope is the payload, defined by a triplet: a **Buffer** (the memory pointer to the data), a **Count** (the number of items), and a **Datatype**. Because MPI needs to know the exact layout of bytes to transmit across the wire, you must explicitly state the type of data being sent using one of the predefined MPI constants shown in **Table 1** below.

Table 1: Common C to MPI Type Mappings

C Data Type	MPI Data Type	Typical Size (Bytes)
`char`	`MPI_CHAR`	1
`int`	`MPI_INT`	4
`long`	`MPI_LONG`	4 or 8 (System Dependent)
`long long`	`MPI_LONG_LONG`	8
`float`	`MPI_FLOAT`	4
`double`	`MPI_DOUBLE`	8

MPI provides two distinct modes for sending and receiving messages, each with different performance implications. The default mode is **Blocking Communication** (e.g., `MPI_Send`). In this mode, the function call acts as a stop sign; it does not return control to the program until the message buffer is "safe" to reuse. This usually means the system has copied the data into a network buffer or successfully transmitted it. While safe and easy to reason about, blocking calls effectively pause the CPU, preventing it from doing useful work while waiting for the network card.

To maximize performance, HPC developers often use **Non-Blocking Communication** (e.g., `MPI_Isend`). These functions return immediately—essentially initiating a "request" to send data in the background. This allows the CPU to continue performing calculations while the data is simultaneously travelling over the network, a technique known as **overlapping computation and communication**. However, this adds complexity: the programmer must explicitly check later (using `MPI_Wait`) to ensure the transfer finished before touching the data buffer again.

3.5 Deadlocks

A **Deadlock** is a catastrophic state where a parallel program hangs indefinitely because two or more processes are waiting for each other to do something, and neither can proceed. This is the parallel programming equivalent of four cars arriving at a 4-way stop sign simultaneously, with no one willing to move.

Consider a scenario where Rank 0 and Rank 1 both try to receive a message from each other at the same time. Rank 0 calls `MPI_Recv` and waits for Rank 1. Simultaneously, Rank 1 calls `MPI_Recv` and waits for Rank 0. Since `MPI_Recv` is a blocking call, both sit there forever, and neither ever reaches the line of code to send the message. To fix this, developers must carefully order their operations (pair every Send with a matching Recv in the correct order) or use combined functions like `MPI_Sendrecv` to handle the exchange atomically.

3.6 Python Parallelism: `mpi4py`

`mpi4py` is a Python library that provides bindings for the standard MPI standard, allowing you to write parallel Python scripts. We will see Python examples side-by-side with C examples in the following sections.

- **Ease of Use**: It handles data serialization (pickling) automatically, so you can send complex Python objects (lists, dictionaries) between nodes.
- **Performance**: For heavy computation, `mpi4py` can wrap C libraries (like NumPy), giving C-like speed with Python syntax.

> *"Note: Detailed discussions on HPC Libraries (including further usage of Python in HPC) will be covered in Week 7."*

---

4. Point-to-Point Communication

Point-to-Point communication is the most basic form of data exchange in MPI, where one specific process sends a message to another specific process. This is analogous to a direct phone call.

4.1 Standard Point-to-Point Functions

C Syntax

`MPI_Send`

```
int MPI_Send(void *buf, int count, MPI_Datatype datatype,
```

Explanation: C function signature for sending a message in MPI. It requires specifying the buffer, count, datatype, destination rank, message tag, and the communicator context.

Argument	Description
buf	Pointer to the data to send.
count	Number of elements.
datatype	MPI data type (e.g., `MPI_DOUBLE`).
dest	Rank of the destination process.
tag	Integer identifier for the message class.
comm	The communicator.

`MPI_Recv`

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype,
```

Explanation: C function signature for receiving a message in MPI. It blocks until a message matching the specified source rank and tag arrives.

Argument	Description
buf	Pointer to the buffer where received data will be stored.
count	Maximum number of elements to receive.
datatype	MPI data type.
source	Rank of the sender (or `MPI_ANY_SOURCE`).
tag	Message tag to match (or `MPI_ANY_TAG`).
comm	The communicator.
status	A structure to store metadata about the actual message received (or `MPI_STATUS_IGNORE`).

WEEK 5 WEEK 7

Python Syntax

```
comm.send(obj, dest, tag=0)
```

Explanation: Python equivalent (mpi4py) for sending a Python object. Unlike C, it handles serialization (pickling) and datatypes automatically.

- **obj**: Python object to send.
- **dest**: Rank of receiver.

`comm.recv`

```
data = comm.recv(source=MPI.ANY_SOURCE, tag=MPI.ANY_TAG,
```

Explanation: Python equivalent for receiving an object. Using `ANY_SOURCE` allows the program to accept a message from whichever rank sends it first.

- **Return**: The received object.

4.2 Code Example: "Ping Pong"

This simple program demonstrates point-to-point communication.

C Implementation:

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);

    int rank;
```

```
    int tag = 0;

    if (rank == 0) {
        number = -1;
        MPI_Send(&number, 1, MPI_INT, 1, tag, MPI_COMM_WC
        printf("Process 0 sent number %d to Process 1\n",
    } else if (rank == 1) {
        MPI_Recv(&number, 1, MPI_INT, 0, tag, MPI_COMM_WC
        printf("Process 1 received number %d from Process
    }

    MPI_Finalize();
    return 0;
}
```

Explanation: Rank 0 prepares an integer (`-1`) and uses `MPI_Send` to push it to Rank 1. Rank 1 executes `MPI_Recv`, blocking until the message arrives. Once the transfer is complete, both processes print a confirmation message and exit.

Python Implementation:

```
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()

if rank == 0:
    data = {'key1': [7, 2.72], 'key2': ('abc', 'xyz')}
    print(f"Rank 0 sending data: {data}")
    comm.send(data, dest=1, tag=11)
elif rank == 1:
```

WEEK 5 WEEK 7

Explanation: Rank 0 creates a complex Python dictionary containing lists and tuples. It sends this object directly using `comm.send`. `mpi4py` automatically pickles (serializes) the object into bytes, transmits it, and unpickles it on Rank 1, where `comm.recv` reconstructs the original dictionary.
Comparison: The logic remains identical (Role check → Send/Recv). However, Python eliminates the need to specify memory buffers, counts, and datatypes, as it can serialize arbitrary objects.

---

5. Advanced MPI & Collective Communication (Sterling Chapter 9)

While Point-to-Point communication is like a 1-on-1 phone call, parallel algorithms often require **Collective Communication**—where everyone talks to everyone (like a conference call).

5.1 Collectives

Collective communication refers to a set of optimized MPI functions where **every process** in a specific communicator participates in the operation. Unlike point-to-point communication, which involves only a sender and a receiver, a collective call implies a global coordination.

Crucially, these functions often act as implicit **synchronization barriers**—processes that arrive early will wait until others catch up. Under the hood, MPI implementations optimize these operations using sophisticated algorithms (e.g., binary tree or hypercube topologies) to minimize network latency and bandwidth usage, far outperforming naive manual implementations.

5.1.1 Broadcast (`MPI_Bcast`)

`MPI_Bcast` replicates data from a single root process to all other processes in the communicator. Rank 0 often reads configuration or initial data and must distribute it to every process needs, such as

and serial), `MPI_Bcast` uses a tree-based algorithm. Rank 0 sends to two nodes, who each send to two others, and so on. This logarithmic scaling $O(\log N)$ allows data to reach thousands of nodes almost instantly.

C Syntax:

```
int MPI_Bcast(void *buffer, int count, MPI_Datatype datat
```

Explanation: C function signature for broadcasting data. The `root` argument defines which rank holds the original data that gets sent to everyone else.

Argument	Description
buffer	Pointer to the data element(s). On Root, it's the source; on others, it's the destination.
count	Number of elements in the buffer.
datatype	The type of data (e.g., `MPI_INT`).
root	Rank of the broadcast root process.
comm	The communicator (e.g., `MPI_COMM_WORLD`).

Python Syntax:

```
data = comm.bcast(data, root=0)
```

Explanation: Python syntax for a broadcast operation. The data variable on the root rank is distributed and returned on all other ranks.

C Example:

```
int data;
if (rank == 0) data = 100;
MPI_Bcast(&data, 1, MPI_INT, 0, MPI_COMM_WORLD);
```

Explanation: Rank 0 initializes `data` to 100. The `MPI_Bcast` function then is called by **all** processes. Rank 0 acts as the root/sender, and every other process acts as a receiver. After this line, every process's `data` variable will contain 100.

Python Example:

```
data = None
if rank == 0:
    data = {'key': 100}
data = comm.bcast(data, root=0)
```

Explanation: In Python, `mpi4py` handles type discovery. Rank 0 creates a dictionary. The `bcast` method distributes this dictionary to all other ranks, who start with `data = None`. After the call, all ranks possess a copy of the dictionary.

5.1.2 Scatter / Gather

Data decomposition is central to parallel processing. `MPI_Scatter` takes a large array residing on the Root process, slices it into equal chunks, and deals one chunk to each process in the communicator.

> *"Note: Data Decomposition (also known as Domain Decomposition) is covered in detail in Week 7."*

C Syntax:

```
int MPI_Scatter(void *sendbuf, int sendcount, MPI_Datatyp
                void *recvbuf, int recvcount, MPI_Datatyp
```

```
int root, MPI_Comm comm)
```

Explanation: C function signature for scattering an array. It splits the `sendbuf` array from the root and distributes the chunks sequentially to all ranks.

Argument	Description
sendbuf	Pointer to the full data array (relevant only on Root).
sendcount	Number of items sent to *each* process (not total).
sendtype	Data type of sent elements.
recvbuf	Pointer to the local buffer where this process's chunk will be stored.
recvcount	Number of elements this process receives (should match `sendcount`).
recvtype	Data type of received elements.
root	Rank of the scattering process.
comm	The communicator.

Python Syntax

```
local_data = comm.scatter(sendbuf, root=0)
```

Explanation: Python syntax for scattering. The root rank provides a list, and each rank receives one element from that list as `local_data`.

C Example

```
int global_data[4] = {10, 20, 30, 40};
int local_data;
```

Explanation: The `global_data` array exists only on Rank 0. `MPI_Scatter` takes this array and sends one integer to each of the 4 processes (assuming `size=4`). Rank 0 gets `10`, Rank 1 gets `20`, and so on. The result is stored in each process's `local_data` variable.

Python Example

```
if rank == 0:
    data = [10, 20, 30, 40]
else:
    data = None
local_data = comm.scatter(data, root=0)
```

Explanation: Rank 0 initializes a standard Python list. The `scatter` method divides the list elements among the processes. Each process receives one element of the list into `local_data`.

`MPI_Gather` is the inverse operation. It takes chunks of data from every process and reassembles them into a single large array on the Root process.

C Syntax

```
int MPI_Gather(void *sendbuf, int sendcount, MPI_Datatype
               void *recvbuf, int recvcount, MPI_Datatype
               int root, MPI_Comm comm)
```

Explanation: C function signature for gathering data. It performs the inverse of scatter, collecting data chunks from all ranks into a single array on the root.

Argument	Description
sendbuf	Pointer to the local

WEEK 5 WEEK 7

Argument	Description
sendcount	Number of elements this process sends.
sendtype	Data type of sent elements.
recvbuf	Pointer to the full array to reconstruct (relevant only on Root).
recvcount	Number of elements received from *each* process (should match `sendcount`).
recvtype	Data type of received elements.
root	Rank of the gathering process.
comm	The communicator.

Python Syntax:

```
all_data = comm.gather(local_data, root=0)
```

Explanation: Python syntax for gathering. Each rank contributes `local_data`, and rank 0 receives a complete list of all contributions.

C Example:

```
int local_result = rank * 2;
int global_results[4];
MPI_Gather(&local_result, 1, MPI_INT, global_results, 1,
```

Explanation: Each process has a `local_result` (e.g., Rank 0→0, Rank 1→2...). `MPI_Gather` collects these integers and places them into the `global_results` array on Rank 0 in rank order: `{0, 2, 4, 6}`.

Python Example:

WEEK 5 WEEK 7

```
local_result = rank * 2
all_data = comm.gather(local_result, root=0)
```

Explanation: `mpi4py` collects the `local_result` from every rank and creates a standard Python list `all_data` on Rank 0 containing `[0, 2, 4, 6]`.

5.1.3 Reduction (`MPI_Reduce`)

Often, the final answer requires combining partial results from every node, such as finding the global maximum temperature or the sum of all local errors.

C Syntax:

```
int MPI_Reduce(void *sendbuf, void *recvbuf, int count, M
               MPI_Op op, int root, MPI_Comm comm)
```

Explanation: C function signature for a reduction operation. It collects data from all ranks and combines them using an operation (e.g., sum, max, min) on the root.

Argument	Description
sendbuf	Pointer to the local data to be combined.
recvbuf	Pointer to the buffer where the result is stored (relevant only on Root).
count	Number of elements to reduce.
datatype	Data type of elements.
op	Operation to perform (e.g., `MPI_SUM`, `MPI_MAX`, `MPI_MIN`).
root	Rank of the process receiving the final result.
comm	The communicator.

WEEK 5 WEEK 7

Python Syntax:

```
global_result = comm.reduce(local_result, op=MPI.SUM, roc
```

Explanation: Python syntax for reduction. It efficiently sums up the `local_result` from all ranks into a single variable on rank 0.

C Example:

```
int local_sum = rank * 10;
int global_sum;
MPI_Reduce(&local_sum, &global_sum, 1, MPI_INT, MPI_SUM,
```

Explanation: Each process calculates a `local_sum` (e.g., Rank 1 has 10, Rank 2 has 20). `MPI_Reduce` collects these values, applies the `MPI_SUM` operation (10+20+...), and stores the final total in `global_sum` only on Rank 0.

Python Example:

```
local_sum = rank * 10
global_sum = comm.reduce(local_sum, op=MPI.SUM, root=0)
```

Explanation: Python's `reduce` works similarly. Each rank computes a value, and they are summed together. The result is returned only to Rank 0's `global_sum` variable; other ranks receive `None`.

5.2 Communicators

While `MPI_COMM_WORLD` includes everyone, sometimes we want to split the cluster into subgroups (e.g., "Row Communicators" and "Column Communicators" for matrix math). `MPI_Comm_split` cr

This example demonstrates a parallel Matrix-Vector multiplication ($y = A \times x$) using a combination of collectives.

- **Goal**: Multiply a 4×4 Matrix (A) by a Vector of size 4 (x). The operation is $y_i = \text{row}_i \cdot x$.

How this demonstrates Collectives: To calculate $y = Ax$, every row of the matrix A needs to be multiplied by the *entire* vector x. This requires a specific pattern of data movement:

1. **Broadcast (`MPI_Bcast`) for Shared Data**: Since every process needs to multiply its row against the *same* vector x, we use `MPI_Bcast`. This efficiently replicates x from the Root to all other processes in $O(\log P)$ time, ensuring everyone has the data they need to compute.

2. **Scatter (`MPI_Scatter`) for Data Decomposition**: The matrix A represents the primary workload. Instead of every process storing the entire matrix (which wastes memory), we use `MPI_Scatter` to break the domain into chunks. Each process receives only **one row** of the matrix. This is a classic example of **Domain Decomposition** (discussed in Week 7).

3. **Local Computation**: Each rank now possesses one row of A and the full vector x. It performs a dot product calculation independently, producing a single scalar value (a partial result).

4. **Gather (`MPI_Gather`) for Result Reassembly**: The final answer y is a vector, but currently, its components are scattered across the cluster (Rank 0 has y_0, Rank 1 has y_1, etc.). `MPI_Gather` collects these distributed scalars and reconstructs the full vector y on the Root process for output.

C Implementation:

```
#include <mpi.h>
#include <stdio.h>
```

WEEK 5 WEEK 7

```
    MPI_Init(&argc, &argv);

    int rank, size;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Assume size=

    // Data buffers
    int A[4][4] = { {1, 1, 1, 1}, {2, 2, 2, 2}, {3, 3, 3,
    int x[4] = {10, 20, 30, 40};
    int local_A[4];     // Buffer for 1 row of A
    int local_y = 0;    // Local result (dot product)
    int y[4];           // Final result vector

    // 1. Broadcast Vector x to everyone
    MPI_Bcast(x, 4, MPI_INT, 0, MPI_COMM_WORLD);

    // 2. Scatter Rows of Matrix A
    // Send 4 integers (1 row) to each process
    MPI_Scatter(A, 4, MPI_INT, local_A, 4, MPI_INT, 0, MP

    // 3. Local Computation: Dot Product
    for (int i = 0; i < 4; i++) {
        local_y += local_A[i] * x[i];
    }
    printf("Rank %d local result: %d\n", rank, local_y);

    // 4. Gather Results into Vector y
    MPI_Gather(&local_y, 1, MPI_INT, y, 1, MPI_INT, 0, MP

    if (rank == 0) {
        printf("Final Result y: [%d, %d, %d, %d]\n", y[0]
    }

    MPI_Finalize();
```

WEEK 5 WEEK 7

Explanation: Rank 0 holds the full matrix. `MPI_Bcast` ensures everyone has the vector `x`. `MPI_Scatter` splits the matrix so Rank 0 processes Row 0, Rank 1 processes Row 1, etc. After computing the local dot product, `MPI_Gather` reassembles the individual scalar results into the final vector `y` on Rank 0.

Python Implementation (with NumPy)

```python
from mpi4py import MPI
import numpy as np

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Data Initialization
if rank == 0:
    A = np.array([[1, 1, 1, 1],
                  [2, 2, 2, 2],
                  [3, 3, 3, 3],
                  [4, 4, 4, 4]], dtype=int)
    x = np.array([10, 20, 30, 40], dtype=int)
else:
    A = None
    x = np.empty(4, dtype=int) # Allocate space for x

# 1. Broadcast Vector x
comm.Bcast(x, root=0)

# 2. Scatter Rows of Matrix A
# Allocate space for 1 row of size 4
local_A = np.empty(4, dtype=int)
# comm.Scatter expects buffers of correct size/type
```

WEEK 5 WEEK 7

```
comm.Scatter(A, local_A, root=0)
# 3. Local Computation: Dot Product
local_y = np.dot(local_A, x)
print(f"Rank {rank} local result: {local_y}")

# 4. Gather Results
if rank == 0:
    y = np.empty(4, dtype=int)
else:
    y = None

# Gather scalar results (using Gather for arrays or gathe
# Since local_y is a scalar from numpy, we can use the ob
# but for high performance we would use comm.Gather with
# Here we use comm.Gather with a 1-element buffer for str
local_y_buf = np.array([local_y], dtype=int)
comm.Gather(local_y_buf, y, root=0)

if rank == 0:
    print(f"Final Result y: {y}")
```

Explanation: Logic mimics the C version. We use **uppercase** methods (`comm.Bcast`, `comm.Scatter`) which are optimized for `numpy` arrays and buffer-like objects, rather than the lowercase pickle-based methods (`comm.bcast`). This is the standard "High Performance" way to use Python for numeric work. Note how we must pre-allocate `x` and `local_A` buffers on the receiving nodes.

---

6. Parallel Strategy & Libraries

6.1 Loading Libraries

WEEK 5 WEEK 7

you use **Environment Modules** to load pre-installed, optimized software stacks. This allows multiple versions of the same software (e.g., Python 3.8 and Python 3.11) to coexist without conflict.

Managing the environment involves a few key commands. You can view all available software on the cluster using `module avail`, which lists the specific versions installed. To add a piece of software to your active environment, use `module load <name>`, which updates your `$PATH` and `$LD_LIBRARY_PATH`. You can check what is currently loaded with `module list` or clear your environment completely with `module purge`.

Example: Setting up for MPI:

```
$ module load openmpi5
$ module load python/3.10
$ module load numpy/1.24
```

Explanation: Environment setup commands showing how to load the required software modules (MPI and Python) before running a parallel job.

After running these, `mpicc` and `python` will be available in your terminal.

6.2 The HPC Software Stack (Sterling Chapter 10)

Rule #1 of HPC: Don't rewrite what experts have already optimized. HPC applications rely on a rich ecosystem of middleware and libraries. Using these standard libraries ensures your code runs as fast as possible (high performance) and can move between different clusters easily (portability).

6.2.1 Numerical Linear Algebra

Linear algebra forms the backbone of scientific computing, and efficiency here is critical. The **Basic Linear Algebra Subprograms (BLAS)** provide the *de facto* standard interface for vector and matrix operations. They are organized into three levels based

operations. Level 3 is particularly crucial for high performance because its $O(N^3)$ computational work allows for excellent cache reuse compared to the $O(N^2)$ data movement.

Example: Matrix Multiplication with CBLAS (Level 3) Instead of writing three nested loops, you call `cblas_dgemm`.

```
// C = alpha*A*B + beta*C
cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasNoTrans,
            M, N, K, alpha, A, K, B, N, beta, C, N);
```

Explanation: An example of a highly optimized BLAS library call for matrix multiplication, which is orders of magnitude faster than a manual nested loop.

Building on top of BLAS, **LAPACK** provides higher-level solvers for complex mathematical tasks such as solving systems of linear equations ($Ax = b$) and Eigenvalue problems. While LAPACK is designed for shared-memory (single node) systems, **ScaLAPACK** extends these capabilities to distributed memory systems using MPI.

6.2.2 Parallel I/O

Reading and writing data can become a major bottleneck at scale. Writing ASCII text files (like CSVs) from thousands of cores is incredibly slow and inefficient in terms of storage. To address this, libraries like **HDF5 (Hierarchical Data Format v5)** and **NetCDF** provide self-describing binary formats for storing large, complex, heterogeneous data. Crucially, they support **Parallel I/O**, allowing thousands of MPI ranks to write to a single file simultaneously without data corruption.

Example: Parallel Write with HDF5

```
// Each MPI rank defines its chunk of the dataset (hyperslab)
H5Sselect_hyperslab(filespace, H5S_SELECT_SET, start, NULL, count, NULL);
// Parallel write using MPI-IO driver
H5Dwrite(dset_id, H5T_NATIVE_INT, memspace, filespace, plist_id, data);
```

Explanation: Demonstrates parallel I/O using HDF5 hyperslabs, where multiple MPI ranks write concurrently to different regions of the same file without conflict.

6.2.3 Fast Fourier Transforms (FFT)

For applications in signal processing and partial differential equations, Fast Fourier Transforms are essential. **FFTW (Fastest Fourier Transform in the West)** is the gold standard library for this purpose. It is an adaptive library that effectively "learns" the fastest way to compute an FFT on your specific hardware architecture at runtime (creating a "plan"), optimizing itself for the cache and processor characteristics of the machine.

Example: Planning an FFT

```
// Create a plan (FFTW measures the system speed)
fftw_plan p = fftw_plan_dft_1d(N, in, out, FFTW_FORWARD, FFTW_MEASURE);
// Execute the transform
fftw_execute(p);
```

Explanation: Demonstrates FFTW's plan creation, an optimization technique where the library measures system hardware characteristics to choose the fastest algorithmic execution path.

6.2.4 Benefits of Libraries

The primary benefits of using these libraries are performance and portability. Hardware vendors (Intel, AMD, NVIDIA) provide highly optimized implementations of standards like BLAS (e.g., Intel MKL, OpenBLAS, cuBLAS). A naive matrix multiplication loop written by hand might achieve only a fraction of the processor's peak performance,

of your source code.

---

7. Summary and Next Steps

This week, we transitioned from building the cluster to programming it. We established the physics of performance (**Amdahl's Law**), learned to utilize all cores in a single box with **OpenMP**, and broke the single-node barrier using **MPI**. You now possess the syntax to send messages, but syntax alone does not solve problems.

Preparing for Week 7

Next week, we move from *Syntax* to *Strategy*. We will explore **Parallel Algorithms**—standard design motifs like **Domain Decomposition**, **Manager-Worker**, and **Halo Exchange**—that dictate *how* we organize our computation and communication to solve real scientific problems efficiently.

Checklist:

- Can you calculate the maximum theoretical speedup of a code that is 5% serial?
- What is the difference between a **Blocking** (`MPI_Send`) and **Non-Blocking** (`MPI_Isend`) call?
- Why does `MPI_Bcast` scale better than a loop of `MPI_Send` calls?
- When would you use **OpenMP** instead of MPI, and vice versa?

Resources

- **Hager, G. & Wellein, G. (2010)**. *Introduction to High Performance Computing for Scientists and Engineers*. CRC Press.
- **Sterling Text**: Chapter 7.2.3

LLNL HPC Tutorial
OpenMP Cheat Sheet
4.5-1115-CPP-web.pdf

Test Your Knowledge

Ready to check your understanding of this week's material? Take the interactive quiz now!

Start Quiz